

Semana 5

O LLM é Só Uma Peça

De construtores do motor a utilizadores do veículo

Semanas 1–4: construíram um GPT do zero.

Hoje: o que acontece quando escalamos — e o que o motor NÃO consegue fazer sozinho.

SGAIM · 2025/2026 · MEIC · ISEL

Fase 1 completa – o que construíram

Semana 1: $p(x)$, manifolds, MNIST	← intuição generativa
Semana 2: autograd + building blocks	← motor de gradientes
Semana 3: self-attention + modelo completo	← arquitetura
Semana 4: treino + geração + ablation	← o motor funciona!

4.192 parâmetros. Algoritmicamente idêntico ao GPT-4.

Sabem o que há **dentro** do motor. Isso muda tudo.

A partir de agora: o motor é uma peça de algo maior.

MP1 — sondagem

- 1 Quem entregou / está a entregar?
- 2 Qual foi o experiment de ablation mais surpreendente?
- 3 Algum resultado que contradisse a vossa previsão?

2 minutos: partilhem com o colega do lado o resultado que mais vos surpreendeu.

A ponte

Na aula 04: *“De construtores do motor a utilizadores do veículo.”*

Hoje vamos:

1

Ver o motor à escala real

Ollama, modelos de 1B–7B parâmetros

2

Descobrir o que o motor NÃO faz

Stateless, sem ferramentas, sem memória

3

Aprender a comunicar com ele

Prompt engineering

PARTE 1

0 motor à escala real

O motor à escala real: Ollama

```
ollama run llama3.2:1b
```

É o **mesmo motor** que construíram:

- Attention (multi-head, causal, com KV cache)
- Residual connections
- RMSNorm
- MLP blocks
- Softmax + temperatura

A diferença é **escala**: 4.192 → 1.000.000.000+ parâmetros.

Modelos disponíveis

Modelo	Parâmetros	RAM (~Q4)	Caso de uso
llama3.2:1b	1.2B	~1 GB	Testes rápidos, pouca RAM
llama3.2:3b	3.2B	~2 GB	Equilíbrio velocidade/qualidade
qwen2.5:7b	7.6B	~5 GB	Melhor qualidade, mais lento

Os três usam a **mesma arquitetura** — attention + residual + MLP + normalization.

A diferença: dimensões maiores, mais layers, mais dados de treino.

THINK

O vosso microGPT tem 4.192 parâmetros e gera nomes como “emma”, “josh”.

Um modelo com 1B parâmetros gera texto coerente em múltiplas línguas.

O que muda na arquitectura?
E o que se mantém exactamente igual?

Pensem 1 minuto.

Escala $\neq$ magia

O que se mantém

- Attention (Q, K, V, Vd)
- Residual connections
- RMSNorm
- MLP (expand + compress)
- Cross-entropy training
- Adam optimizer

O que muda

- n_embd: 16 $\rightarrow$ 2048–4096
- n_layer: 1 $\rightarrow$ 24–32
- n_head: 4 $\rightarrow$ 32–64
- vocab: 27 $\rightarrow$ 32k–128k
- Dados: 32k chars $\rightarrow$ triliões de tokens
- + SFT + RLHF (aula 04)

A arquitetura é a mesma. Vocês compreendem como funciona por dentro.

A maioria dos utilizadores de LLMs não. Vocês sim.

Quantização

1B parâmetros $\times$ 16 bits = **2 GB** de RAM. 7B parâmetros $\times$ 16 bits = **14 GB** de RAM.

Solução: representar cada peso com menos bits.

Formato	Bits/peso	RAM (7B)	Qualidade
FP16	16	~14 GB	Referência
Q8	8	~7 GB	Quase idêntica
Q4	4	~4 GB	Ligeira degradação
Q2	2	~2 GB	Degradação notável

Trade-off: menos RAM e mais velocidade $\leftrightarrow$ perda de qualidade.
Os modelos Ollama vêm tipicamente em Q4 por defeito.

Hands-on — parte 1: escala e quantização

Objectivo: ganhar intuição sobre escala e trade-offs. Trabalhem em pares.

1 Mesma pergunta a 1b e 7b. Comparem qualidade.

2 Raciocínio multi-passo (ex: céu azul). Onde falha o 1B?

3 Meçam tempo até 1.^a palavra e tempo total. Diferença?

4 Código Python (ex: inverter string). Qual é mais fiável?

Registem observações — vamos discutir.

PARTE 2

O que o LLM NÃO consegue

O que o LLM NÃO consegue

Sabem o que o motor faz: receber tokens → produzir distribuição → amostrar.

Mas o motor tem limitações fundamentais:

- ✗ Não tem memória entre conversas
- ✗ Não sabe que dia é hoje
- ✗ Não acede à internet
- ✗ Não faz cálculos com precisão
- ✗ Não pode executar código
- ✗ Não pode verificar os seus próprios factos

O LLM é uma função stateless: input → output. Nada mais.

Stateless: sem memória

Conversa 1:

user: "O meu nome é Ana."

model: "Olá Ana! Como posso ajudar?"

Conversa 2:

user: "Qual é o meu nome?"

model: "Não tenho acesso a essa informação."

**Entre chamadas,
o modelo esquece
tudo. Não há
estado persistente.**

A “memória” que o ChatGPT parece ter é uma ilusão

O **sistema** copia as mensagens anteriores para o prompt seguinte.

O modelo não lembra. O sistema à volta do modelo é que gere a memória.

A context window: memória de trabalho

Tudo o que o modelo “sabe” está na **context window** — a sequência de tokens que recebe como input.

```
system prompt + mensagens anteriores + msg actual
```

```
← tudo o que o modelo vê
```

Fora da context window: **não existe.**

Modelo	Context window
llama3.2	128k tokens (~100 páginas)
qwen2.5	128k tokens
GPT-4o	128k tokens
Claude	200k tokens

THINK

Peçam ao modelo: “Que dia é hoje?”

O que responde? Porque não consegue responder correctamente?

Identifiquem duas razões diferentes para esta limitação.

Pensem 1 minuto.

Duas razões, dois problemas

1. Knowledge cutoff

O modelo só sabe o que estava nos dados de treino.
Eventos depois da data de corte não existem para ele.

Solução: **RAG** (semanas 7–8)

2. Sem ferramentas

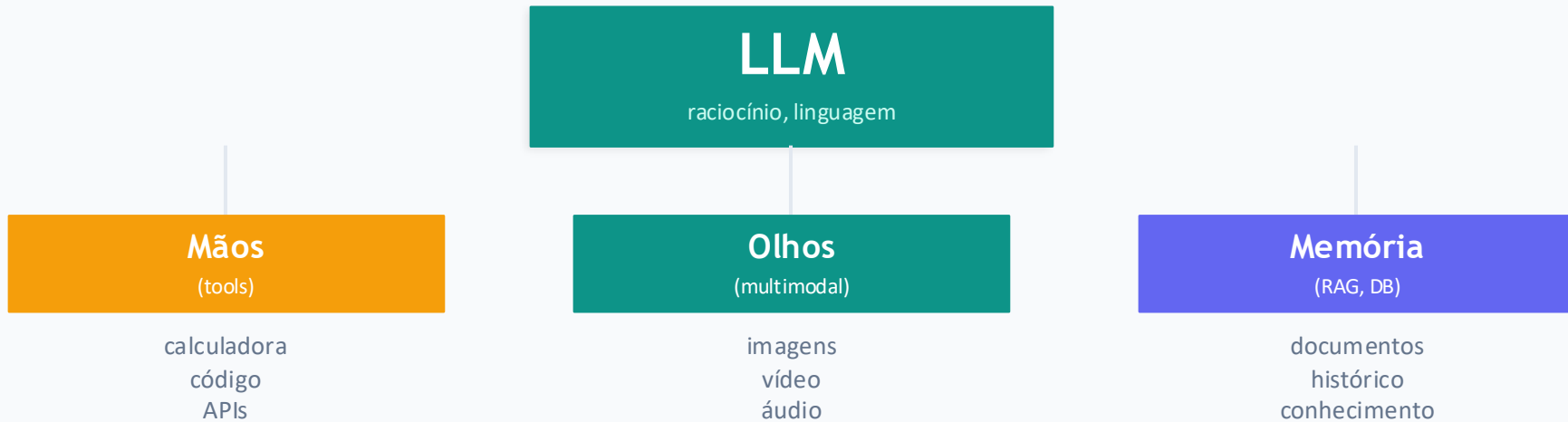
Mesmo que soubesse a data do treino, não tem relógio,
não tem calendário, não pode executar `datetime.now()`.

Solução: **Tool use** (semana 6)

São problemas **diferentes** com soluções **diferentes**.

O LLM como cérebro

O LLM é poderoso — mas sozinho, é como um **cérebro sem corpo**:



A **Fase 2** do curso: dar mãos, olhos e memória ao LLM.

Mapa da Fase 2

Semana	Tema	O que adicionamos
5 (hoje)	O LLM é só uma peça	Compreensão das limitações
6	Structured output + tool use	Mãos — o LLM pode agir
7	RAG — retrieval	Memória — consultar documentos
8	RAG — evaluation	Medir se a memória funciona

De uma peça isolada a um sistema inteligente.

Na **Fase 3** (semanas 9–12): damos-lhe autonomia — **agentes**.

PARTE 3

Prompt Engineering

A única interface: o prompt

Não podem mudar o motor (pesos fixos após treino).

Mas podem mudar

como comunicam com ele — o **prompt**.

Prompt engineering não é “hack” — é a interface entre o humano e o modelo.

4 técnicas fundamentais:

- 1 Zero-shot** Pedir directamente, sem exemplos
- 2 Few-shot** Dar exemplos no prompt
- 3 Chain-of-thought** Obrigar a pensar passo a passo
- 4 System prompts** Definir papel e restrições

Zero-shot: pedir directamente

Classifica o seguinte texto como POSITIVO ou NEGATIVO:
"O filme foi uma perda de tempo."

O modelo responde com base **apenas no conhecimento do treino**.

Sem exemplos, sem formato definido.

Funciona para
tarefas simples e comuns.

Falha em
tarefas ambíguas ou com formato específico.

Few-shot: dar exemplos

Classifica o sentimento:

"Adorei o concerto" → POSITIVO

"Péssimo serviço" → NEGATIVO

"Nada de especial" → NEUTRO

"O filme foi uma perda de tempo" →

Exemplos no prompt

Ensinam o formato
e calibram o modelo.

Não mudam os pesos —
mudam o
contexto.

É como dar 3 problemas resolvidos antes do exercício. O modelo extrapola o padrão.

THINK

O modelo não muda os pesos durante uma conversa.

Então como é que “aprende” com os exemplos no prompt?

Pensem no mecanismo de attention — o que faz o modelo com os tokens dos exemplos?

Pensem 1 minuto.

In-context learning

Os exemplos não mudam os pesos. Mudam a **distribuição de atenção**.

O modelo “vê” os exemplos como parte da sequência de input. O mecanismo de attention — que construíram no ex3 — distribui pesos sobre os tokens do contexto.

Com exemplos de **texto** → **POSITIVO**, o modelo atende ao padrão e extrapola para o novo input.

É o mesmo attention que implementaram. Não há magia — é auto-regressão com contexto mais rico.

In-context learning: aprender sem treinar, só com contexto.

Chain-of-thought: “pensa passo a passo”

Sem CoT

"Quanto é 17×24 ?"
→ "408" (ou "412"...)

Com CoT

"Quanto é 17×24 ?
Pensa passo a passo."
→ " $17 \times 20 = 340$. $17 \times 4 = 68$.
 $340 + 68 = 408$."

Chain-of-thought obriga o modelo a gerar passos intermédios.

Cada passo torna-se contexto para o seguinte — o modelo “usa” a sua própria geração como memória de trabalho temporária.

Quando usar: raciocínio multi-passo, matemática, lógica, problemas com sub-etapas.

System prompts: definir o papel

```
system: "És um professor de física. Respondes sempre com  
analogias do dia-a-dia. Se o aluno der uma resposta  
errada, faz-lhe uma pergunta em vez de corrigir."
```

```
user: "O que é a gravidade?"
```

O **system prompt** é prepended ao início da context window — invisível ao utilizador final, mas visível ao modelo.

Define: **tom, personalidade, restrições, formato.**

O AI tutor que usaram no MP1 é um system prompt socrático.

Hands-on — parte 2: prompt engineering

Objectivo: ganhar fluência em técnicas de prompt.

5

Zero-shot vs few-shot

Classificar frases (desporto, política, tecnologia).
Sem exemplos, depois com 3 exemplos. Comparem.

6

Chain-of-thought

Pergunta de lógica. Com e sem
“pensa passo a passo”.

7

System prompt

Use `/set system` no Ollama.
Criem assistente com personalidade.

8

Hallucination

Encontrem algo que o modelo inventa
com confiança. Expliquem porquê.

Wrap-up

Hoje:

- O motor à escala real (Ollama) — mesma arquitectura, diferente escala
- Limitações fundamentais: stateless, sem ferramentas, sem memória
- Prompt engineering: zero-shot, few-shot, CoT, system prompts

A lição central

O LLM é um motor poderoso mas incompleto. Para construir sistemas inteligentes, precisamos de mais do que um bom motor — precisamos de um veículo completo.

Semana 6 + TPC

TPC

- Experimentar prompts diferentes com Ollama — registrar o que funciona e o que não funciona
- Tentar "quebrar" o system prompt — dar instruções que o modelo deveria recusar
- Explorar: ollama list, ollama show <modelo>, ollama ps

Semana 6: Structured output + Tool use

- JSON mode, constrained generation
- Function calling — o LLM ganha "mãos"
- O loop de orquestração:
- LLM → ferramenta → resultado → LLM

O veículo começa a ganhar forma.